

Nxt Trendz

E-Commerce Shopping Experience

Case Study Overview

Nxt Trendz is a responsive e-commerce storefront built to demonstrate a complete authenticated shopping journey. The experience takes a user from login through product discovery, filtering, product evaluation, and cart management.

The project focuses on the parts of a storefront that create trust and momentum: clear navigation, useful discovery tools, feedback during network requests, graceful failure states, and a cart that remains available when the user returns.

The Challenge

A product catalog can become difficult to use when customers have to scan a large list without meaningful search or filtering. A shopping experience also needs to handle more than the successful API response. Slow requests, empty results, authentication boundaries, and unavailable products are all normal states that need deliberate UI treatment.

The goal was to build a storefront that:

- Protects shopping routes behind authentication.
- Helps users find products through search, category, rating, and price sorting.
- Makes product evaluation possible before adding an item to the cart.
- Keeps cart interactions simple and persistent.
- Communicates loading, empty, and failure states clearly.
- Works across desktop and mobile layouts.

Product Experience

1. Sign In

Users authenticate through the login API. The returned JWT is stored in a cookie and used for subsequent requests. Users who try to access a protected route without a token are redirected to the login page.

2. Discover Products

The Products page combines exclusive Prime Deals with the full catalog. Users can search by title, select a category, choose a minimum rating, and sort by price.

Filters are composed into the same catalog request, so a user can narrow the catalog with multiple conditions instead of choosing only one search path.

3. Evaluate a Product

Each product card links to a dedicated product details route. The details view provides the product image, title, brand, price, rating, review count, availability, description, quantity controls, and similar products.

4. Add and Manage Cart Items

Users select a quantity before adding an item. The cart merges repeated additions of the same product, supports quantity changes, removes items when their quantity reaches zero, and calculates the current subtotal.

Cart data is synchronized with localStorage, allowing the shopping bag to survive navigation and later visits in the same browser.

Solution Architecture

The application uses a route-oriented component structure:

```
App
|-- LoginForm
|-- ProtectedRoute
|   |-- Home
|   |-- Products
|       |-- PrimeDealsSection
|       |-- AllProductsSection
|           |-- FiltersGroup
|           |-- ProductsHeader
|           |-- ProductCard
|       |-- ProductItemDetails
|       |-- SimilarProductItem
|-- Cart
```

Authentication Boundary

ProtectedRoute checks for the jwt_token cookie before rendering protected pages. The header owns logout behavior and removes the token before redirecting the user to login.

API State Handling

Data-fetching components use explicit status values for INITIAL, IN_PROGRESS, SUCCESS, and FAILURE. This makes each network state visible in the UI and provides a retry action when a request fails.

The catalog also distinguishes an empty successful response from a failed request, allowing users to understand whether their filters returned no products or whether the service was unavailable.

Data Normalization

API fields such as image_url are mapped to the UI model's imageUrl before product data is passed into reusable presentation components. This keeps API-specific naming details at the data boundary.

Client-Side Cart Persistence

The cart is intentionally lightweight for this project. Product and quantity data are stored in localStorage, while the Cart component maintains the rendered state. Updates write to both locations so the UI and browser storage remain synchronized.

Key Technical Decisions

Decision	Reason
React Router protected routes	Keeps authentication logic at the navigation boundary.
Cookie-based JWT storage	Matches the API's authenticated request model.
Explicit API status constants	Makes loading, success, empty, and failure views predictable.
Shared ProductCard component	Keeps catalog, deals, and similar products visually consistent.
URL query parameters for filters	Lets the API perform catalog filtering and supports combined criteria.
localStorage cart	Provides persistence without requiring a separate cart backend.
Responsive CSS layouts	Supports the same shopping flow across viewport sizes.

Challenges and Solutions

Keeping Multiple Filters in Sync

Search, category, rating, and sorting all affect the same catalog request. The active values are held together in the catalog component and used to construct one request, ensuring the current selection is not lost when another filter changes.

Separating Empty and Failure States

An empty product array is a valid response, while a non-OK response indicates a request problem. The UI renders a no-products view for the first case and a retryable failure view for the second.

Maintaining Cart Consistency

Adding an existing product should increase its quantity rather than create a duplicate line item. Cart updates therefore check the product ID, merge quantities when needed, and remove entries when a quantity reaches zero.

Designing for Narrow Screens

The navigation has separate desktop and mobile presentations, and the shopping views use responsive layout rules so product browsing and cart management remain usable on smaller screens.

Validation Approach

The repository includes the following quality workflows:

```
npm test
npm run lint
npm run build
```

The main scenarios to validate are:

- Unauthenticated access redirects to /login.
- Successful login redirects to the home page.
- Product search triggers on Enter.
- Category, rating, and sort selections update the catalog request.
- Empty results render the no-products view.
- Failed requests render a retry action.
- Product quantity cannot decrease below one on the details page.
- Adding the same product again increases its cart quantity.
- Cart quantity updates persist to localStorage.

Outcome

The finished project demonstrates a complete front-end commerce workflow rather than an isolated product list. It combines authentication, protected navigation, REST API integration, reusable UI components, responsive styling, asynchronous state handling, and browser persistence in one coherent experience.

Resume-Ready Description

Built a responsive React e-commerce storefront with JWT authentication, protected routes, REST API integration, multi-criteria product discovery, sorting, loading and error states, product detail pages, similar-product recommendations, and a persistent local-storage shopping cart.

Future Improvements

- Add a backend-backed cart and checkout flow.
- Add wishlist functionality for authenticated users.
- Move API URLs and environment-specific settings into environment variables.
- Add automated component and integration coverage for the primary shopping journeys.
- Add pagination or infinite scrolling for larger product catalogs.
- Add cart count feedback directly in the navigation.